LLM MODEL
FINE-TUNING

BY: VANN VISAL

TABLE OF CONTENT

01 Fine-Tuning

02 Model & Preprocessor Setup

03 Data Preparation

04 Supervised Trainer Setup

05 Training Diagnosis

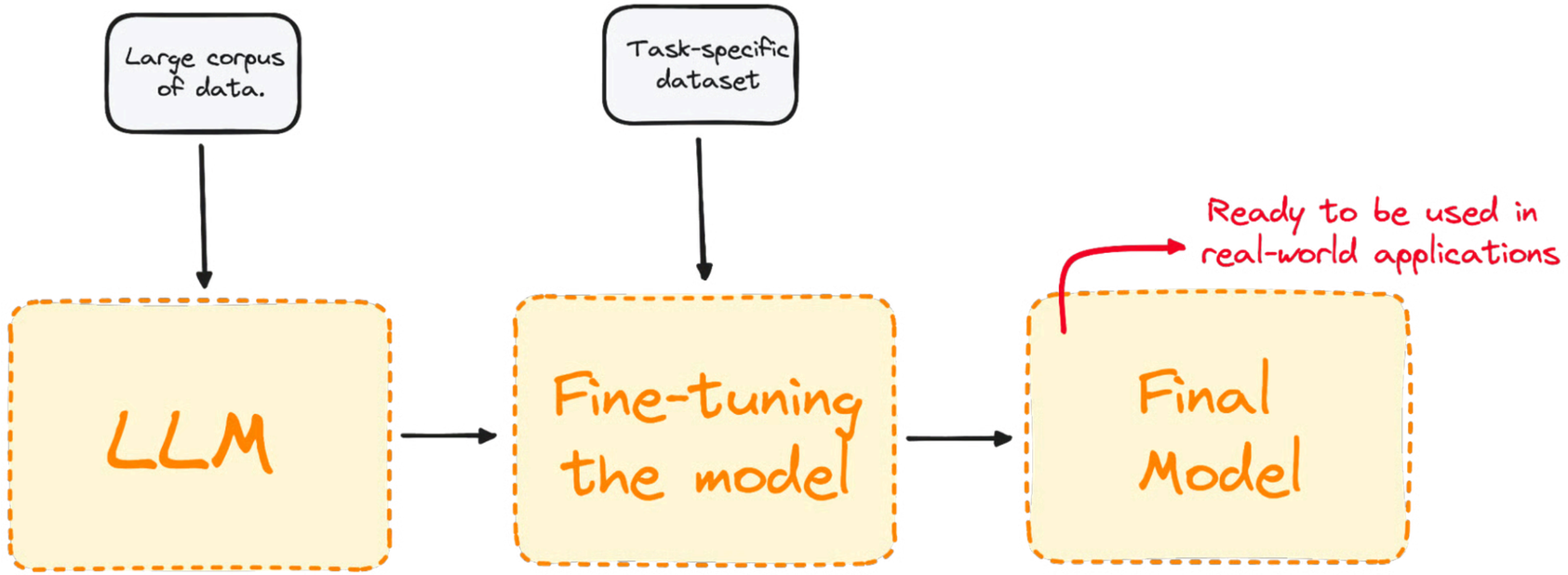
06 Model Checkpoints

Fine-Tuning

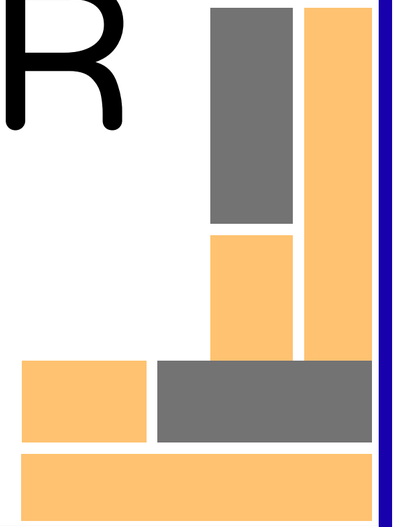


FINETUNING

Fine-tuning is a technique where a **pretrained model** is adapted to a new, specific task by continuing its training on a smaller, task-specific dataset. It leverages the knowledge learned from large-scale datasets and adjusts the model's parameters to improve performance on the new task.



MODEL & PREPROCESSOR



MODEL SELECTION

Factors in Model Selection

Task Alignment

Choosing models based on task type and pretraining

Model Architecture

Evaluating architecture suitability and modularity

Model Size and Complexity

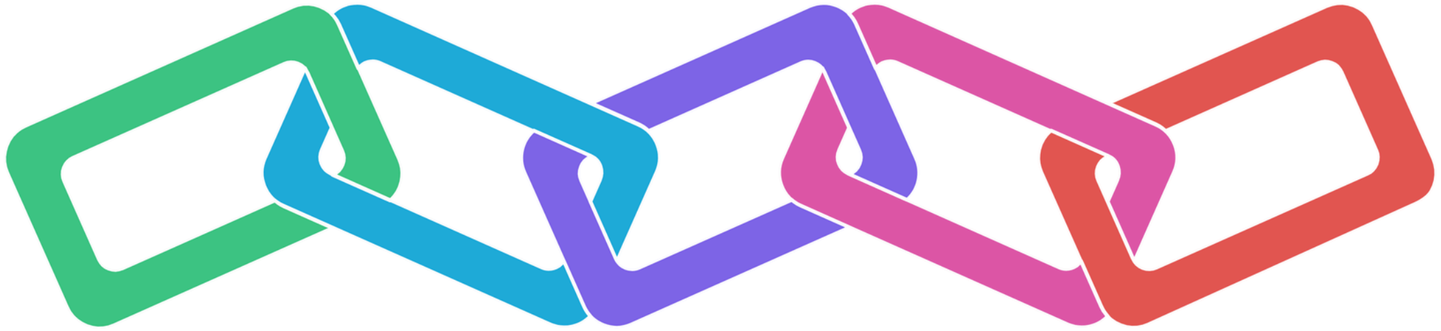
Balancing model size with available resources

Performance Benchmarking

Assessing existing performance and transfer learning

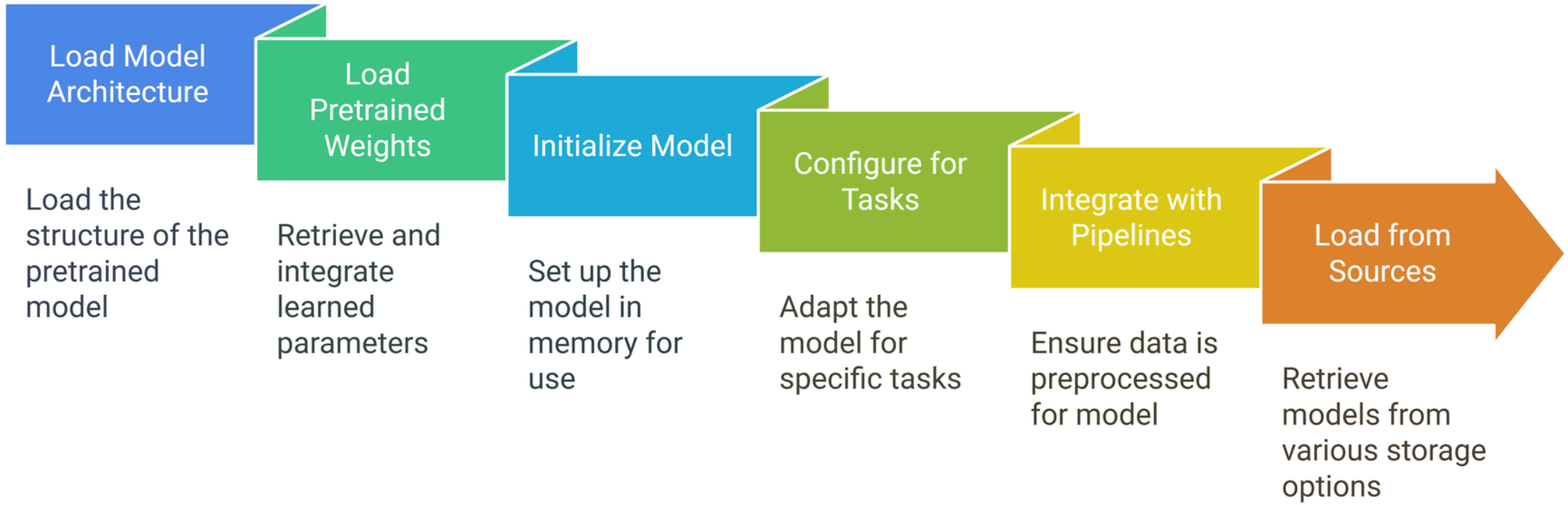
Community Support and Documentation

Ensuring documentation and framework support



MODEL LOADER

A model loader is a utility or function that allows you to load a pretrained model, its architecture, and the associated weights (parameters) into memory so that you can use it for inference or further training (fine-tuning).



MODEL LOADER

In Hugging Face's **Transformers** library, there are two main categories of model loaders: **Model-specific loaders** and **AutoModel loaders**.

1. Model-Specific Loaders

- Definition: These loaders are tied to a specific model architecture. They are used when you know the exact type of model you want to load (e.g., BERT, GPT-2, T5).

- Example:
  - BertForSequenceClassification.from_pretrained('bert-base-uncased')
  - GPT2LMHeadModel.from_pretrained('gpt2')
  - T5ForConditionalGeneration.from_pretrained('t5-small')

MODEL LOADER

2. AutoModel Loaders

- Definition: The AutoModel class and its variants automatically select the correct model architecture based on the model identifier you provide. This loader abstracts away the need to specify the model architecture explicitly, making it easier to switch between different model types.

- Example:
  - AutoModelForSequenceClassification.from_pretrained('bert-base-uncased')
  - AutoModelForTokenClassification.from_pretrained('roberta-base')
  - AutoModelForCausalLM.from_pretrained('gpt2')

MODEL LOADER

Summary of `AutoModel` Variants

AutoModel Loader	Description
`AutoModel`	General-purpose model, no specific task or head attached.
`AutoModelForSequenceClassification`	For models trained for sequence classification tasks (e.g., text classification).
`AutoModelForTokenClassification`	For models used in token classification tasks (e.g., NER).
`AutoModelForQuestionAnswering`	For models designed for question answering tasks.
`AutoModelForCausalLM`	For models trained for autoregressive text generation (e.g., GPT).
`AutoModelForMaskedLM`	For models trained for masked language modeling (e.g., BERT).
`AutoModelForSeq2SeqLM`	For sequence-to-sequence tasks (e.g., T5, BART for translation or summarization).
`AutoModelForSpeechSeq2Seq`	For speech-to-text models (e.g., ASR).
`AutoModelForVision`	For models trained on visual tasks like image classification.
`AutoModelForImageSegmentation`	For image segmentation tasks.

PROCESSOR LOADER

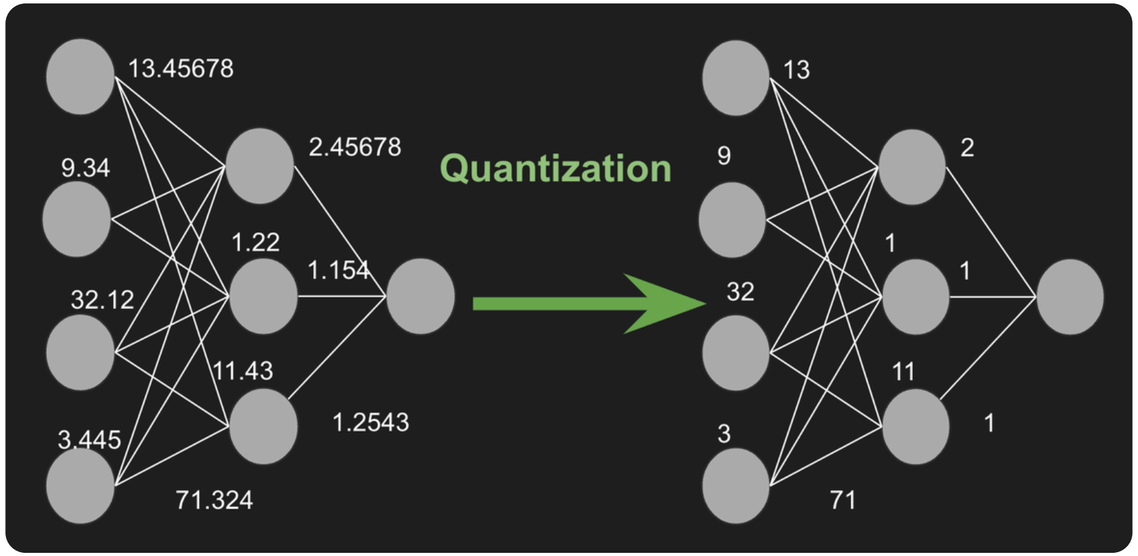
Similar to **AutoModel,** the **AutoProcessor** class is used for automatically selecting the appropriate processor based on the type of model you are using. Different AutoProcessor classes are used for different types of models (text, image, audio, etc.), and they combine the functionalities of tokenizers, feature extractors, and image processors, depending on the model's input type**.**

Processor Types include: Tokenizer, Image Processor, Audio Processor, Multimodal Processor and Audio-Text Processor.

```
model_name = "Qwen/Qwen2-VL-2B-Instruct"
processor = Qwen2VLProcessor.from_pretrained(model_name, use_fast=True)
#or
processor = AutoProcessor.from_pretrained(model_name, use_fast=True)
```

QUANTIZATION

Quantization is the process of reducing the precision of the weights or activations in a neural network model. This technique is mainly used to make models smaller, faster, and more efficient, the model's parameters (weights) are often stored as 32-bit floating-point numbers (FP32). Quantization reduces the precision of these values, using lower-bit representations like 8-bit integers (INT8) or 16-bit floating-point (FP16).



QUANTIZATION

The configuration parameters within this **BitsAndBytesConfig** help you define how quantization should be applied:

1. **load_in_8bit=True**: specifies that the model should be loaded with 8-bit precision for its weights
2. **bnb_8bit_use_double_quant=True**: apply an additional layer of quantization to improve the efficiency and performance of the 8-bit representation

```python
from transformers import BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(
                                load_in_8bit=True,
                                bnb_8bit_use_double_quant=True,
                                bnb_8bit_quant_type="nf4",
                                bnb_8bit_compute_dtype=torch.float16
)
```

3. **bnb_8bit_quant_type="nf4"**: defines the quantization type used when converting the model's weights
4. **bnb_8bit_compute_dtype=torch.float16**: specifies the data type used for computations during the forward pass

MODEL SETUP

When loading Model, we can either use quanitzation config or load_in_4bit hyperparameter for quantization.

```
model = AutoModelForImageTextToText.from_pretrained(
    model_name1,

    quantization_config=quantization_config,            # use quantization config
    # or
    load_in_4bit=True, # 4bit quantization with defualt setting

    torch_dtype=torch.float16,   # weight dtype, overwritten by quantization
    device_map="auto",
    local_files_only = True, # only load if model downloaded locally
    force_download=True, # force redownloading everytime
    resume_download = True, # resume unfinished download

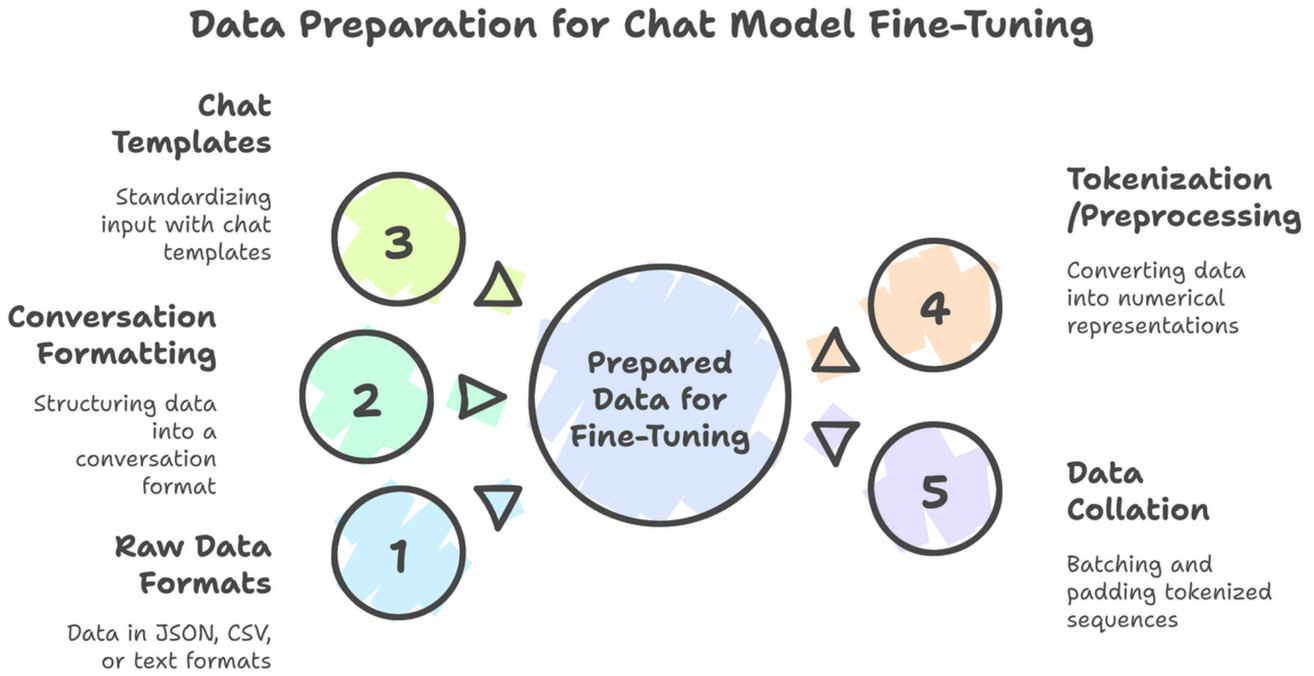
    attn_implementation="flash_attention_2" # if applicable, depend on model
)
```

Data Preparation



PREPARATION STAGES

Fine-tuning a model requires properly structured data, ensuring compatibility with the tokenizer, model architecture, and training pipeline. Below are the key stages:



RAW FORMATS

The data flow for fine-tuning a chat model begins with preparing the dataset in a structured format. Initially, raw data—often in JSON, CSV, or text format. In example:


```
{
    "image":"Robot_Arm_Data\/vid_1\/frame_0008.jpg",
    "prompt":"Move the apple closer to me.",
    "answer":"[{\"target\": \"apple\", \"position\": [1.151,-0.4669,0.69139], \"orientation\": [-0.094752,0.66099,0.6787,0.30574]}]"
},
```


The actual formatting here is preferential but its best to structure in a way easily retrieve each feature, such as **key-value json format** which can be loaded using **load_dataset**, since we need to convert to conversation format in the next step.

```
from datasets import load_dataset

dataset = load_dataset('json', data_files={'train': 'Object_Detection/dataset_train.json', 'test': 'Object_Detection/dataset_test.json'})
```

CONVERSATION FORMATS

The conversation format is essential for organizing multi-turn dialogue data in a structured way, allowing the model to understand the flow of interactions between users and assistants. It also provides a structured format so templates can correctly map messages into the model's expected input structure.

```
def convert_to_conversation(sample):
    conversation = [
        { "role": "user",
          "content" : [
            {"type" : "text",  "text"  : sample['prompt']},
            {"type" : "image", "image" : Image.open(sample["image"]).resize((854, 480)).convert('RGB')} ]
        },
        { "role" : "assistant",
          "content" : [
            {"type" : "text",  "text"  : sample["answer"]} ]
        },
    ]
    return { "messages" : conversation }
pass
```

To Identify format: Check HuggingFace Model Desc, Print Chat Template or Check Training Dataset

CHAT TEMPLATES

The apply_chat_template() converts conversation-format into a single text string format specific to chat model's expected input format, it is required to be able to correctly process and generate conversational responses.

```python
messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image",
                "image": "Object_Detection/train/000000037156_jpg.rf.e82bdff0271070984b897c5c1433ecd6.jpg",
            },
            {"type": "text", "text": "Find me bounding box of a bird in this image"},
        ],
    }
]

text = processor.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True
)
```

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
<|vision_start|><|image_pad|><|vision_end|>Find me bounding box of a bird in this image<|im_end|>
<|im_start|>assistant
```

DATA COLLATIONS

A Data Collator in Hugging Face prepares batches of data before feeding them into a model. It handles:

- Tokenization (if not already tokenized)
- Padding (ensuring all inputs in a batch have the same length)
- Special formatting (e.g., adding masks, labels for training)
- Applying Chat Templates (only for some specific collators)

Some data collators can be imported from specific library/framework such as transformers, unsloth, or can be manually defined in a function. We don't manually use collator but instead pass it to a trainer as arguments.

```
from unsloth.trainer import UnslothVisionDataCollator

data_collator = UnslothVisionDataCollator(model, tokenizer)
```

Supervised Trainer Setup



PEFT

PEFT (Parameter-Efficient Fine-Tuning) is a technique that fine-tunes only a small subset of model parameters instead of the entire model. This makes training faster, less memory-intensive, and more efficient, especially for large language models (LLMs).

PEFT Method	How It Works	Best Use Case	Trainable Parameters
LoRA (Low-Rank Adaptation)	Adds **low-rank matrices** to transformer layers instead of modifying weights.	Fine-tuning large LLMs with minimal memory	🟢 Very Few (~0.1%-1%)
Adapters	Inserts small **bottleneck layers** between transformer layers.	Modifying specific knowledge in a model	🟡 Moderate (~1%-5%)
Prefix Tuning	Keeps model weights frozen but **prepends trainable tokens** to input prompts.	Task-specific adaptation (e.g., summarization)	🟢 Few (~0.1%-1%)
P-Tuning (Prompt Tuning)	Uses a small **trainable embedding vector** instead of changing model parameters.	Very low-resource settings, prompt engineering	🔵 Minimal (<0.1%)
IA3 (Sparse Adapters)	Applies **element-wise scaling** to transformer activations.	Even more memory-efficient than LoRA	🟢 Very Few (~0.1%-1%)

Learn More

LORA

LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning (PEFT) method that freezes the original model weights and adds small trainable low-rank matrices to transformer layers. LoRA decomposes weight updates into two small matrices A and B (rank "r" << full model size), so fewer parameters need to be updated.

- Original Weight Matrix: **W** of size (512 × 512)
- LoRA Decomposition: Introduces two smaller matrices:
  - **A** of size (512 × 8) (low-rank dimension r = 8)
  - **B** of size (8 × 512)

The LoRA update is computed as:

$$W' = W + AB$$

where:

$$AB = [(512 \times 8) \times (8 \times 512) = 512 \times 512]$$

Instead of updating **262,144** parameters (512 × 512), we now train only **8,192** parameters (512 × 8 + 8 × 512), saving 97% of trainable parameters!

LORA COFIGS

```python
from unsloth import FastVisionModel

model = FastVisionModel.get_peft_model(
    model,
    finetune_vision_layers     = True, # False if not finetuning vision layers
    finetune_language_layers   = True, # False if not finetuning language layers
    finetune_attention_modules = True, # False if not finetuning attention layers
    finetune_mlp_modules       = True, # False if not finetuning MLP layers

    r = 16,           # The larger, the higher the accuracy, but might overfit
    lora_alpha = 16,  # Recommended alpha == r at least
    lora_dropout = 0.05, # Dropout Chance
    bias = "none", # Also add and train bias to LoRA
    random_state = 3407, # For consistent Initialization
    use_rslora = True,  # Rank stabilized LoRA
    # target_modules = "all-linear" or something like ["q_proj", "v_proj", "o_proj", 'k_proj', "mlp.gate_proj", "mlp.down_proj"]
)

model.print_trainable_parameters()
```

✓ 2.3s

```
Unsloth: Making `model.base_model.model.visual` require gradients
trainable params: 28,950,528 || all params: 2,237,936,128 || trainable%: 1.2936
```

LORA COFIGS

To identify target module names for LoRA, use print(model):

```
Qwen2VLForConditionalGeneration(
  (visual): Qwen2VisionTransformerPretrainedModel(
    (patch_embed): PatchEmbed(
      (proj): Conv3d(3, 1280, kernel_size=(2, 14, 14), stride=(2, 14, 14), bias=False)
    )
    (rotary_pos_emb): VisionRotaryEmbedding()
    (blocks): ModuleList(
      (0-31): 32 x Qwen2VLVisionBlock(
        (norm1): LayerNorm((1280,), eps=1e-06, elementwise_affine=True)
        (norm2): LayerNorm((1280,), eps=1e-06, elementwise_affine=True)
        (attn): VisionSdpaAttention(
          (qkv): Linear4bit(in_features=1280, out_features=3840, bias=True)
          (proj): Linear4bit(in_features=1280, out_features=1280, bias=True)
        )
        (mlp): VisionMlp(
          (fc1): Linear4bit(in_features=1280, out_features=5120, bias=True)
          (act): QuickGELUActivation()
          (fc2): Linear4bit(in_features=5120, out_features=1280, bias=True)
        )
      )
    )
    (merger): PatchMerger(
      (ln_q): LayerNorm((1280,), eps=1e-06, elementwise_affine=True)
      (mlp): Sequential(
        (0): Linear(in_features=5120, out_features=5120, bias=True)
...
```

TRAINER CONFIGS

```python
from trl import SFTTrainer, SFTConfig

args = SFTConfig(
        per_device_train_batch_size = 2,  # Number of samples per device (GPU/CPU) per batch
        gradient_accumulation_steps = 4,  # Number of steps to accumulate gradients before updating weights

        warmup_steps = 5,  # Steps for gradual learning rate increase at the start of training
        # max_steps = 30,  # Uncomment to stop training after a specific number of steps
        num_train_epochs = 1,  # Number of full training passes over the dataset (use this instead of max_steps)

        learning_rate = 2e-4,  # Initial learning rate for optimization
        fp16 = not is_bf16_supported(),  # Use 16-bit floating point precision (if bfloat16 isn't supported)
        bf16 = is_bf16_supported(),  # Use bfloat16 if supported (saves memory on newer GPUs)

        logging_steps = 5,  # Log training metrics every 5 steps
        save_steps = 50,  # Save model checkpoint every 50 steps

        optim = "adamw_8bit",  # Optimizer choice: AdamW in 8-bit precision (memory efficient)
        weight_decay = 0.01,  # Weight decay to prevent overfitting (L2 regularization)
        lr_scheduler_type = "linear",  # Learning rate scheduler (linear decay)

        seed = 3407,  # Random seed for consistency on different runs
        output_dir = "Qwen2b-Obj-Detection-Unsloth",  # Directory to save trained model checkpoints
        report_to = "wandb",  # Report training metrics to Weights & Biases (WandB)

        # Mandatory settings for vision fine-tuning:
        remove_unused_columns = False,  # Ensures compatibility with custom data collators
        dataset_text_field = "",  # Not used for vision fine-tuning
        dataset_kwargs = {"skip_prepare_dataset": True},  # Skips dataset preparation step (faster loading)
        dataset_num_proc = 4,  # Number of CPU processes for data loading
        max_seq_length = 2048,  # Maximum sequence length for tokenized inputs
    )
```

TRAINER

After setting up the configs, we can start fine-tuning by passing the configs, models, processor, data collator, train and evaluation(optional) dataset to the trainer:

```
FastVisionModel.for_training(model) # Enable for training!

trainer = SFTTrainer(
    model = model,  # The model being fine-tuned
    tokenizer = tokenizer,  # Tokenizer for processing input text
    data_collator = UnslothVisionDataCollator(model, tokenizer),  # Must use for vision finetuning!
    args=args, # SFT Configs
    train_dataset = converted_dataset_train,  # Training dataset
)
```

call trainer.train() to start

TRAINING DIAGNOSIS



Telephone: 010 686 878

Email: info@aifarm.dev

LOSS & VAL LOSS

When fine-tuning a model, tracking Loss and Validation Loss is crucial for understanding how well the model is learning and generalizing.

1. Training Loss (Loss)

- It indicates how well the model is minimizing the difference between its predictions and the ground truth.
- A decreasing training loss usually means the model is learning.
- However, an extremely low training loss could indicate overfitting, meaning the model memorized the training data rather than learning general patterns.

2. Validation Loss

- Unlike training loss, validation loss tells us how well the model generalizes to unseen data.
- If validation loss is much higher than training loss, it suggests overfitting—the model is performing well on training data but poorly on new data.
- If both training and validation loss are high, the model may be underfitting, meaning it hasn't learned enough.

WEIGHT & BIAS REPORT

Weights & Biases (WandB) is a popular tool for tracking, visualizing, and analyzing machine learning experiments, including fine-tuning models. When you set report_to = "wandb" in your trainer configuration, it automatically logs various training metrics to your WandB dashboard.

```
import wandb

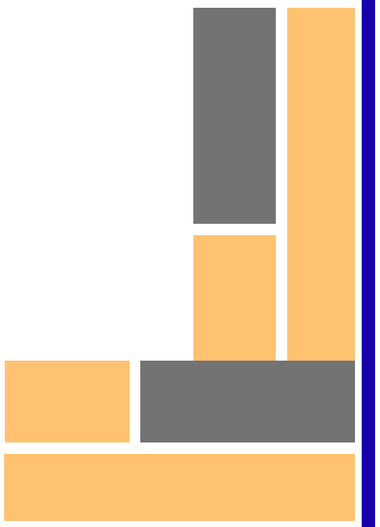
wandb.init(
    project="Qwen2-VL-2B-ObjDetection-Unsloth",  # change this
    name="Qwen2-VL-2B-ObjDetection-Unsloth-Model1",  # change this
)
```

Initialize **wandb** before trainer.train(), it will prompt for login on first use only.

WEIGHT & BIAS REPORT



MODEL CHECKPOINTS



CHECKPOINTS

A **model checkpoint** is a saved state of a model at a specific point during training. It includes the model's weights, optimizer state, and sometimes additional metadata like training step and loss. Checkpoints allow you to:

- Resume training if interrupted.
- Save intermediate progress to avoid retraining from scratch.
- Evaluate models at different stages to select the best version.
- Deploy trained models for inference.

```
model, tokenizer = FastVisionModel.from_pretrained(
    "Qwen2b-Obj-Detection-Unsloth/checkpoint-1000",
    load_in_4bit = True,
)
```

or

```
model, tokenizer = FastVisionModel.from_pretrained(
    "unsloth/Qwen2-VL-2B-Instruct-bnb-4bit", # base model
    load_in_4bit = True,
)

model.load_adapter("Qwen2b-Obj-Detection-Unsloth/checkpoint-1000")
```

We can load Checkpoint directly or apply checkpoint adapter on the base model it is trained on.

Thank you

